

Classifying CIFAR 10 Images Using a Convolutional Neural Network

Justin Ashton

MTH 3320

Introduction to Machine Learning

Dr. Margaret Rooney

Spring 2026

```
In [1]: # Import libraries and classes that will be used
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn

from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical

from sklearn.metrics import classification_report, confusion_matrix
```

Problem Definition

The goal of this project is to build a convolutional neural network (CNN) that can classify CIFAR 10 images into their correct object categories. Each image belongs to one of 10 possible classes, so this is a supervised multi class classification problem.

This project focuses on image classification because CNNs are designed to work well with image data. Instead of looking at the image as one long list of pixel values, a CNN can learn visual patterns such as edges, shapes, colors, and textures. These patterns can then be used to help predict the correct class label.

I expected the model to perform reasonably well, but I also expected some classes to be harder to classify than others. Since CIFAR 10 images are small, some objects can look similar, especially classes like cats, dogs, birds, and deer.

Dataset Description

The dataset used for this project is the CIFAR 10 dataset. It contains small color images that belong to 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Each image is 32 by 32 pixels and has 3 color channels.

The dataset has 50,000 training images and 10,000 testing images. CIFAR 10 is a good dataset for this project because it is more challenging than a simple grayscale image dataset. The images are small and sometimes blurry, so the CNN has to learn useful visual patterns in order to separate the classes correctly.

```
In [2]: # Import dataset
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
```

```
In [3]: # Look at the shape of the training and testing data
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (50000, 32, 32, 3)
y_train shape: (50000, 1)
X_test shape: (10000, 32, 32, 3)
y_test shape: (10000, 1)
```

```
In [4]: # Create class names for the CIFAR 10 dataset
class_names = [
    "airplane",
    "automobile",
    "bird",
    "cat",
    "deer",
    "dog",
    "frog",
    "horse",
    "ship",
    "truck"
]
```

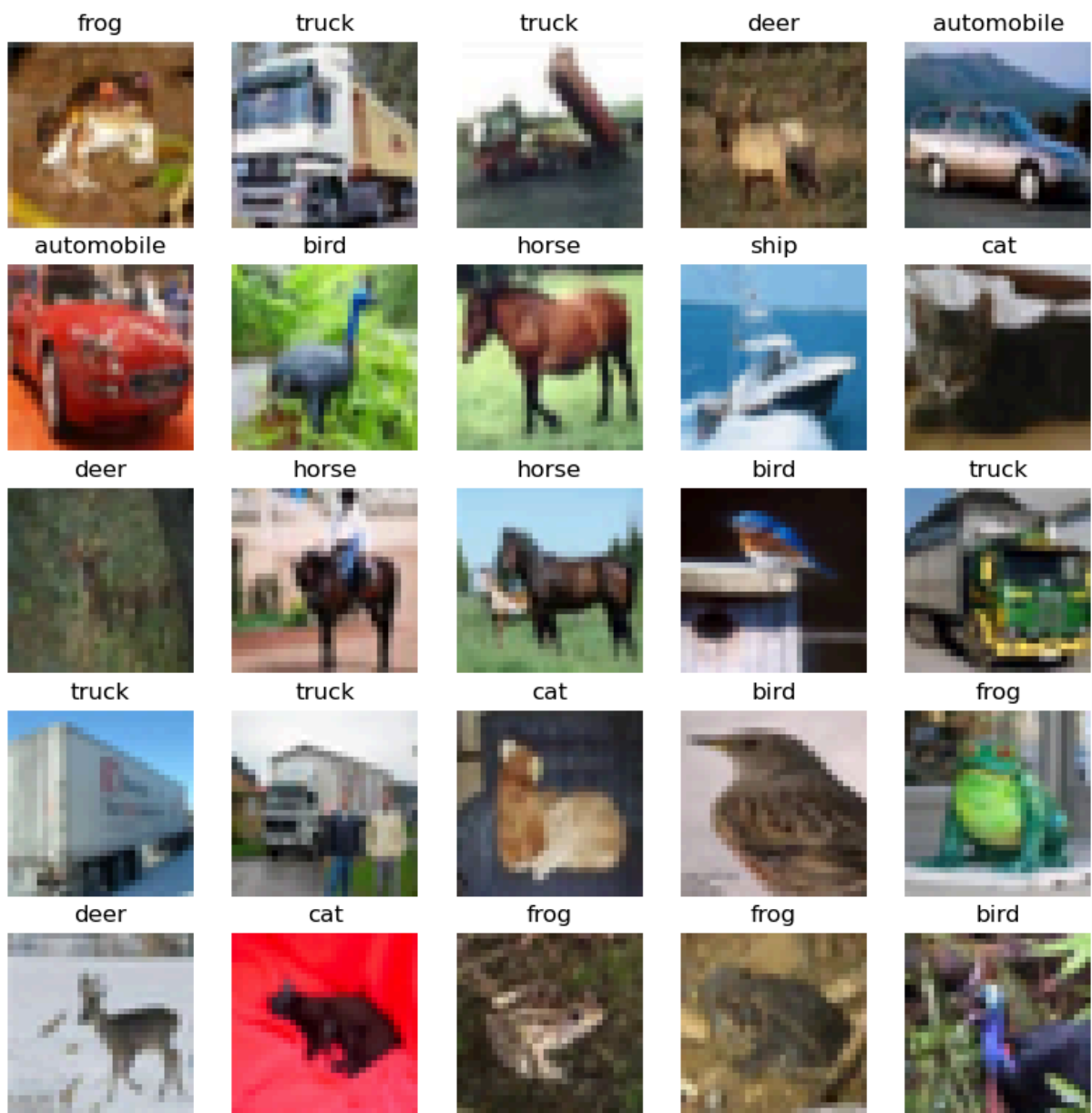
```
In [5]: # Look at the first few target labels
print(y_train[:10])
```

```
[[6]
 [9]
 [9]
 [4]
 [1]
 [1]
 [2]
 [7]
 [8]
 [3]]
```

```
In [6]: # Look at some sample images from the dataset
plt.figure(figsize = (10,10))

for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.imshow(X_train[i])
    plt.title(class_names[y_train[i][0]])
    plt.axis("off")

plt.show()
```



Data Processing

Before training the models, the image pixel values were normalized. The original pixel values range from 0 to 255, so each value was divided by 255 to scale the data between 0 and 1. This helps the neural network train more effectively because the input values are smaller and more consistent.

The target labels were also converted into categorical form because there are 10 possible output classes. This allows the model to treat the task as a multi class classification problem.

```
In [7]: # Normalize the pixel values. Dividing by 255 scales them to between 0 and 1
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
In [8]: # Convert target labels to categorical form
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

```
In [9]: # Look at the original label and the categorical version
print("Original label:", y_train[0])
print("Categorical label:", y_train_cat[0])
```

Original label: [6]

Categorical label: [0. 0. 0. 0. 0. 0. 1. 0. 0. 0.]

CNN Model 1

```
In [10]: cnn_model_1 = Sequential()

#Input Layer
cnn_model_1.add(Input(shape = (32, 32, 3)))

# First convolution and pooling layers
cnn_model_1.add(Conv2D(32, (3, 3), activation = "relu",))
cnn_model_1.add(MaxPooling2D((2, 2)))

#Second convolution and pooling layers
cnn_model_1.add(Conv2D(64, (3, 3), activation = "relu"))
cnn_model_1.add(MaxPooling2D((2, 2)))

# Flatten the feature maps before passing them into dense layers
cnn_model_1.add(Flatten())

# Dense Layer
cnn_model_1.add(Dense(64, activation = "relu"))

# Output Layer
cnn_model_1.add(Dense(10, activation = "softmax"))

# Compile the model
cnn_model_1.compile(
    optimizer = "adam",
    loss = "categorical_crossentropy",
    metrics = ["accuracy"]
)

# Look at the structure of the model
cnn_model_1.summary()
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 64)	147,520
dense_1 (Dense)	(None, 10)	650

Total params: 167,562 (654.54 KB)

Trainable params: 167,562 (654.54 KB)

Non-trainable params: 0 (0.00 B)

CNN Model 1 Description

CNN Model 1 was built as the first model for this project. It was designed as a basic convolutional neural network with 2 convolutional layers, 2 pooling layers, a flatten layer, a dense hidden layer, and 1 output layer.

The convolutional layers are used to detect visual patterns in the images. These patterns may include edges, shapes, colors, and textures that help the model tell the classes apart. The pooling layers reduce the size of the feature maps, which makes the model more efficient while still keeping important information.

After the convolution and pooling layers, the flatten layer converts the feature maps into one long vector. This allows the information to be passed into the dense layer. The final output layer has 10 neurons because there are 10 possible CIFAR 10 classes. The softmax activation function is used because the model needs to choose one class out of the 10 possible options.

```
In [11]: # Train 1st CNN model
history = cnn_model_1.fit(
    X_train,
    y_train_cat,
    epochs = 10,
    batch_size = 64,
    validation_split = 0.2
)
```

Epoch 1/10

625/625 ————— 41s 52ms/step - accuracy: 0.4202 - loss: 1.5980 - val_accuracy: 0.4983 - val_loss: 1.3717

Epoch 2/10

625/625 ————— 24s 38ms/step - accuracy: 0.5556 - loss: 1.2580 - val_accuracy: 0.5903 - val_loss: 1.1662

Epoch 3/10

625/625 ————— 23s 37ms/step - accuracy: 0.6058 - loss: 1.1221 - val_accuracy: 0.5991 - val_loss: 1.1622

Epoch 4/10

625/625 ————— 45s 43ms/step - accuracy: 0.6439 - loss: 1.0213 - val_accuracy: 0.6286 - val_loss: 1.0526

Epoch 5/10

625/625 ————— 24s 38ms/step - accuracy: 0.6654 - loss: 0.9580 - val_accuracy: 0.6429 - val_loss: 1.0171

Epoch 6/10

625/625 ————— 22s 35ms/step - accuracy: 0.6846 - loss: 0.9024 - val_accuracy: 0.6548 - val_loss: 0.9947

Epoch 7/10

625/625 ————— 20s 32ms/step - accuracy: 0.7020 - loss: 0.8546 - val_accuracy: 0.6665 - val_loss: 0.9650

Epoch 8/10

625/625 ————— 22s 34ms/step - accuracy: 0.7180 - loss: 0.8157 - val_accuracy: 0.6763 - val_loss: 0.9416

Epoch 9/10

625/625 ————— 22s 34ms/step - accuracy: 0.7311 - loss: 0.7738 - val_accuracy: 0.6825 - val_loss: 0.9334

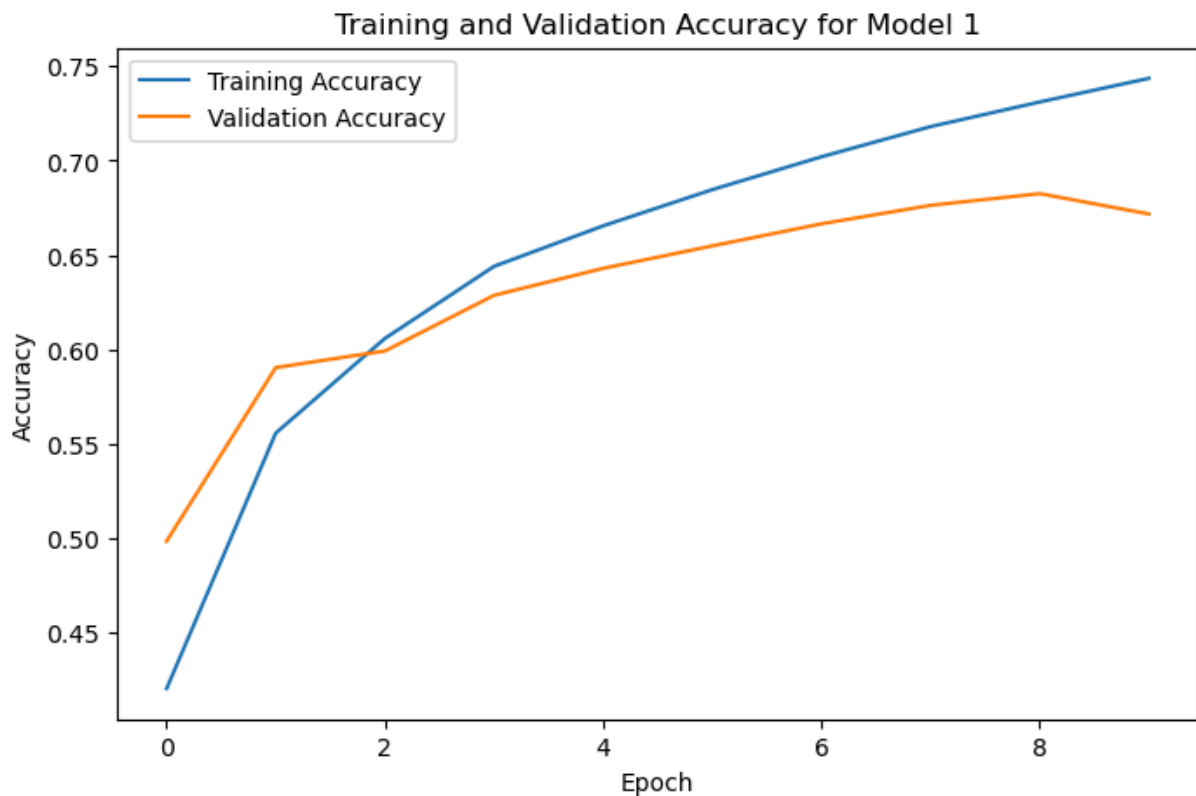
Epoch 10/10

625/625 ————— 23s 36ms/step - accuracy: 0.7436 - loss: 0.7377 - val_accuracy: 0.6717 - val_loss: 1.0070


```
In [12]: # Plot training and validation accuracy for Model 1
plt.figure(figsize = (8,5))

plt.plot(history.history["accuracy"], label = "Training Accuracy")
plt.plot(history.history["val_accuracy"], label = "Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy for Model 1")
plt.legend()
plt.show()
```



Training and Validation Accuracy for CNN Model 1

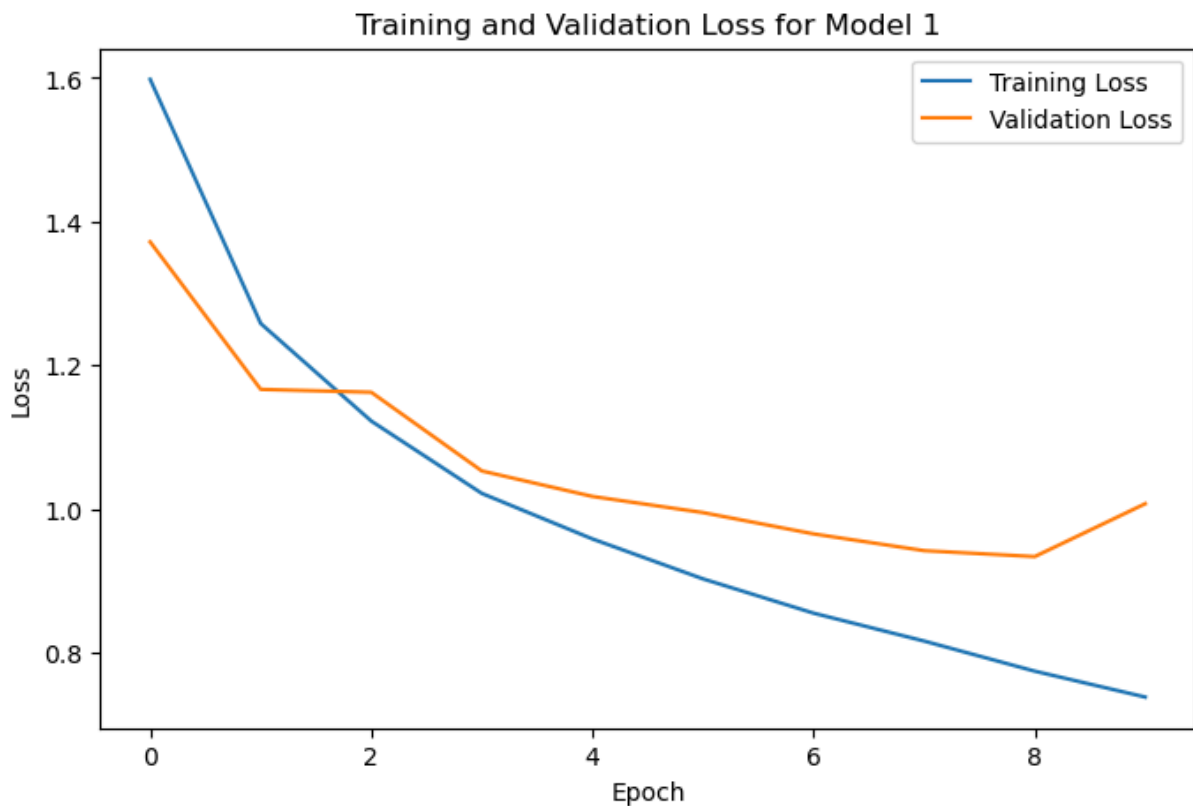
The accuracy graph shows how CNN Model 1 performed during training. The training accuracy increased over the 10 epochs, which shows that the model was learning from the training images.

The validation accuracy also improved, but it started to level off near the later epochs. Since the training accuracy continued to increase while the validation accuracy improved more slowly, this suggests that the model may have started to overfit the training data.

```
In [13]: # Plot training and validation loss for Model 1
plt.figure(figsize = (8, 5))

plt.plot(history.history["loss"], label = "Training Loss")
plt.plot(history.history["val_loss"], label = "Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss for Model 1")
plt.legend()
plt.show()
```



Training and Validation Loss for CNN Model 1

The loss graph gives another view of how CNN Model 1 learned during training. The training loss decreased steadily, which means the model was making fewer errors on the training data.

The validation loss also decreased at first, but it started to level off near the later epochs. This supports the pattern shown in the accuracy graph. CNN Model 1 improved during training, but the gap between training performance and validation performance suggests possible overfitting.

Because of this, I created a second CNN model with dropout to see if regularization could help the model generalize better.

```
In [14]: # Evaluate the 1st model on the test set
test_loss_1, test_accuracy_1 = cnn_model_1.evaluate(X_test, y_test_cat)

print(f"Model 1 Test Loss: {test_loss_1:.4f}")
print(f"Model 1 Test Accuracy: {test_accuracy_1:.4f}")
```

313/313 ————— 3s 10ms/step - accuracy: 0.6664 - loss: 1.0081
Model 1 Test Loss: 1.0081
Model 1 Test Accuracy: 0.6664

CNN Model 1 Test Results

After training, CNN Model 1 was evaluated on the test set. The test set contains images that were not used during training or validation, so it gives a better idea of how the model performs on new data.

CNN Model 1 achieved a test accuracy of about 66.6% and a test loss of about 1.0081. This means the model correctly classified about two thirds of the test images.

The classification report shows that the model performed better on some classes than others. It did better on classes like ship, automobile, frog, horse, and truck. It struggled more with classes like cat, bird, and dog.

```
In [15]: # Predict on test set
y_pred_prob = cnn_model_1.predict(X_test)

# Convert predicted probabilities into class labels
y_pred = np.argmax(y_pred_prob, axis = 1)

# Convert y_test into a one dimensional array
y_test_labels = y_test.flatten()

# Evaluate model performance using classification report
print(classification_report(y_test_labels, y_pred, target_names = class_names))
```

313/313	3s 10ms/step			
	precision	recall	f1-score	support
airplane	0.70	0.64	0.67	1000
automobile	0.74	0.84	0.79	1000
bird	0.57	0.59	0.58	1000
cat	0.58	0.41	0.48	1000
deer	0.68	0.55	0.61	1000
dog	0.67	0.53	0.59	1000
frog	0.85	0.63	0.73	1000
horse	0.64	0.79	0.71	1000
ship	0.60	0.89	0.71	1000
truck	0.69	0.80	0.74	1000
accuracy			0.67	10000
macro avg	0.67	0.67	0.66	10000
weighted avg	0.67	0.67	0.66	10000

CNN Model 2

```
In [17]: cnn_model_2 = Sequential()

#Input layer
cnn_model_2.add(Input(shape = (32, 32, 3)))

#First convolution and pooling layers
cnn_model_2.add(Conv2D(32, (3, 3), activation = "relu",))
cnn_model_2.add(MaxPooling2D((2, 2)))

# Second convolution and pooling layer
cnn_model_2.add(Conv2D(64, (3, 3), activation = "relu"))
cnn_model_2.add(MaxPooling2D((2, 2)))

# Second convolution layer
cnn_model_2.add(Conv2D(64, (3, 3), activation = "relu"))

# Flatten the feature maps before passing them into dense layers
cnn_model_2.add(Flatten())

# Dense layer
cnn_model_2.add(Dense(64, activation = "relu"))
cnn_model_2.add(Dropout(0.5))

# Output layer
cnn_model_2.add(Dense(10, activation = "softmax"))

# Compile the 2nd CNN model
cnn_model_2.compile(
    optimizer = "adam",
    loss = "categorical_crossentropy",
    metrics = ["accuracy"]
)

# Look at the structure of the model
cnn_model_2.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d_2 (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_3 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_4 (Conv2D)	(None, 4, 4, 64)	36,928
flatten_1 (Flatten)	(None, 1024)	0
dense_2 (Dense)	(None, 64)	65,600
dropout (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 10)	650

Total params: 122,570 (478.79 KB)

Trainable params: 122,570 (478.79 KB)

Non-trainable params: 0 (0.00 B)

CNN Model 2 Description

CNN Model 2 was created as an attempt to improve Model 1. It uses 3 convolutional layers, 2 pooling layers, a flatten layer, 1 dense hidden layer, a dropout layer, and 1 output layer.

The extra convolutional layer gives the model another chance to learn visual patterns before the data reaches the dense layer. The dropout layer was also added to help reduce overfitting. During training, dropout randomly ignores some neurons, which can make the model less dependent on specific neurons and more generalizable.

CNN Model 2 has fewer trainable parameters than CNN Model 1 even though it has an extra convolutional layer. This happens because the third convolutional layer reduces the size of the feature maps before they reach the dense layer.

The purpose of this model was to see if adding depth and dropout would improve performance compared to the first model.

```
In [18]: # Train the 2nd CNN model
history_2 = cnn_model_2.fit(
    X_train,
    y_train_cat,
    epochs = 10,
    batch_size = 64,
    validation_split = 0.2
)
```

Epoch 1/10

625/625 ————— 40s 43ms/step - accuracy: 0.3290 - loss: 1.7989 - val_accuracy: 0.4316 - val_loss: 1.5675

Epoch 2/10

625/625 ————— 26s 42ms/step - accuracy: 0.4661 - loss: 1.4696 - val_accuracy: 0.5238 - val_loss: 1.3207

Epoch 3/10

625/625 ————— 25s 40ms/step - accuracy: 0.5281 - loss: 1.3210 - val_accuracy: 0.5707 - val_loss: 1.2116

Epoch 4/10

625/625 ————— 25s 40ms/step - accuracy: 0.5723 - loss: 1.2149 - val_accuracy: 0.6206 - val_loss: 1.0794

Epoch 5/10

625/625 ————— 27s 44ms/step - accuracy: 0.5972 - loss: 1.1438 - val_accuracy: 0.6481 - val_loss: 1.0162

Epoch 6/10

625/625 ————— 23s 37ms/step - accuracy: 0.6186 - loss: 1.0842 - val_accuracy: 0.6561 - val_loss: 0.9882

Epoch 7/10

625/625 ————— 21s 34ms/step - accuracy: 0.6411 - loss: 1.0325 - val_accuracy: 0.6572 - val_loss: 0.9716

Epoch 8/10

625/625 ————— 21s 34ms/step - accuracy: 0.6513 - loss: 0.9931 - val_accuracy: 0.6733 - val_loss: 0.9295

Epoch 9/10

625/625 ————— 20s 33ms/step - accuracy: 0.6658 - loss: 0.9535 - val_accuracy: 0.6731 - val_loss: 0.9407

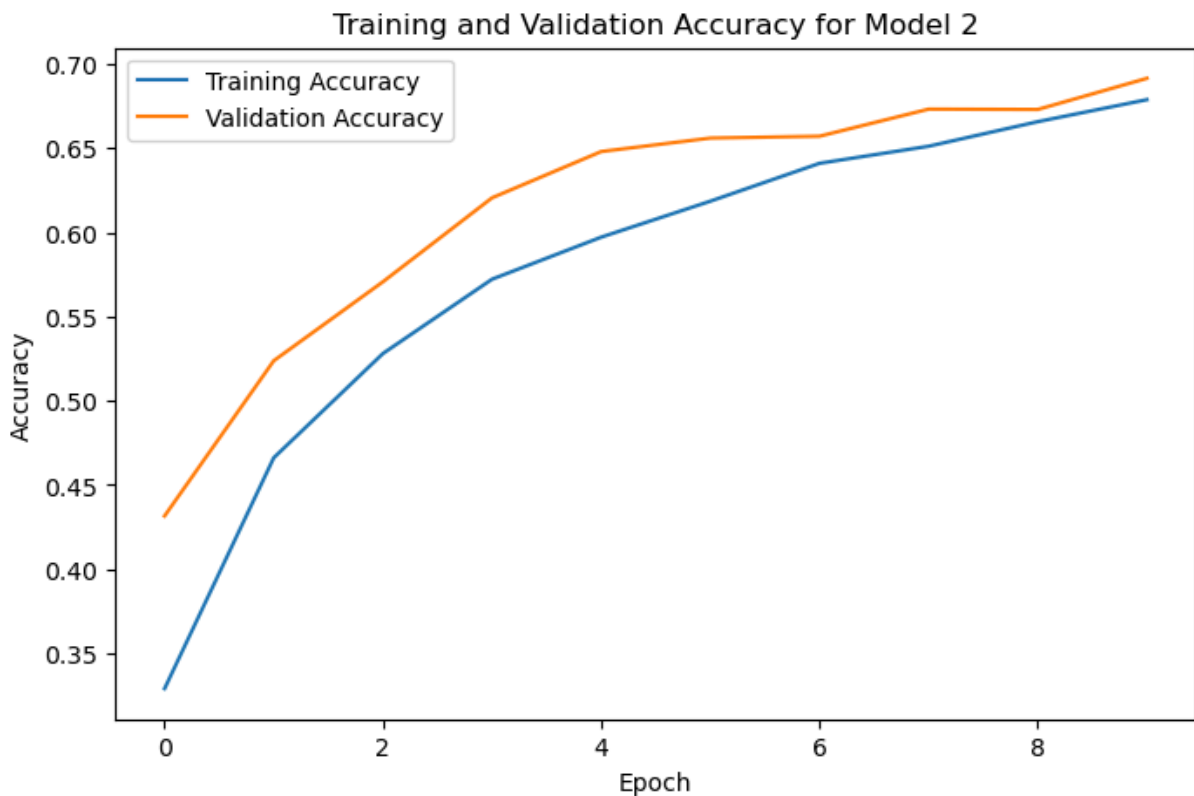
Epoch 10/10

625/625 ————— 21s 34ms/step - accuracy: 0.6789 - loss: 0.9197 - val_accuracy: 0.6916 - val_loss: 0.8837


```
In [19]: # Plot training and validation accuracy for Model 2
plt.figure(figsize = (8,5))

plt.plot(history_2.history["accuracy"], label = "Training Accuracy")
plt.plot(history_2.history["val_accuracy"], label = "Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy for Model 2")
plt.legend()
plt.show()
```



Training and Validation Accuracy for CNN Model 2

The accuracy graph shows that CNN Model 2 improved steadily over the 10 epochs. Both the training accuracy and validation accuracy increased as training continued.

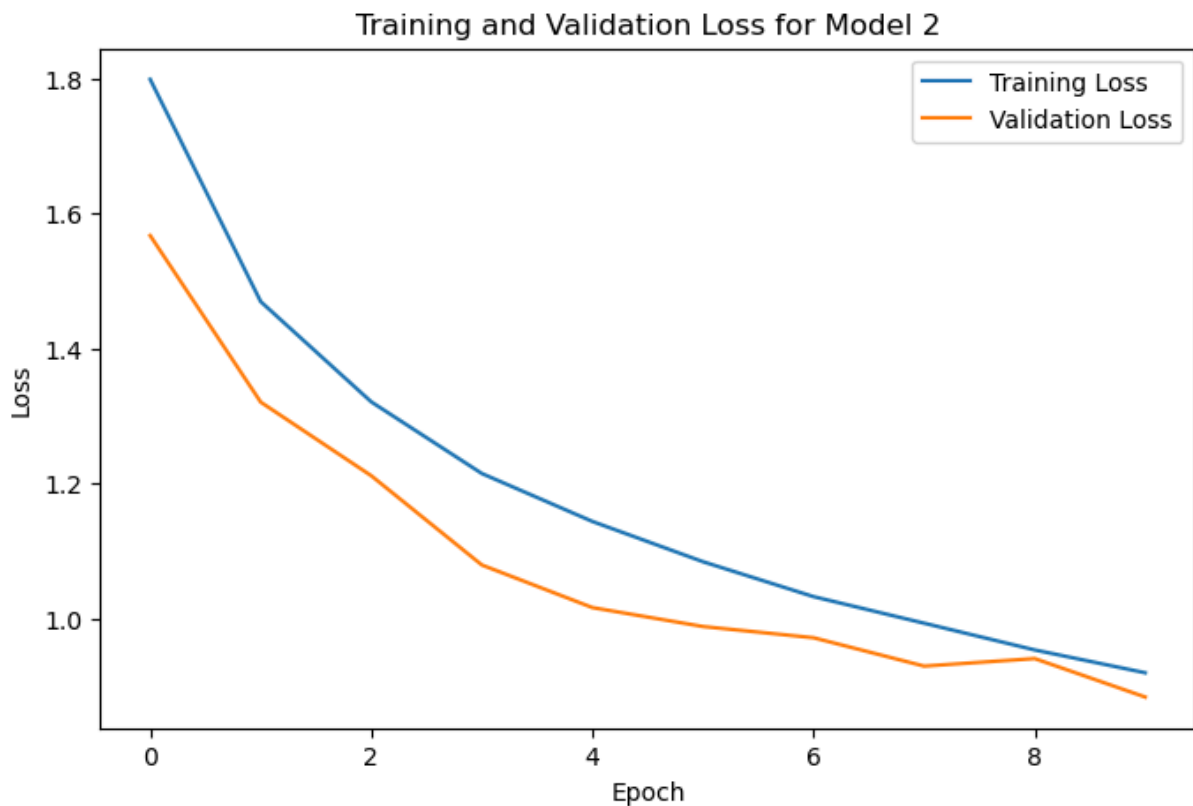
The validation accuracy stayed close to the training accuracy, which suggests that the model was generalizing better than Model 1. This is important because Model 2 was designed to reduce overfitting by using dropout.

Compared to Model 1, the training accuracy was lower, but this is expected because dropout makes training harder. The goal of dropout is not always to increase training accuracy, but to help the model perform better on data it has not seen before.

```
In [20]: # Plot training and validation loss for Model 2
plt.figure(figsize = (8, 5))

plt.plot(history_2.history["loss"], label = "Training Loss")
plt.plot(history_2.history["val_loss"], label = "Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss for Model 2")
plt.legend()
plt.show()
```



Training and Validation Loss for CNN Model 2

The loss graph shows that both the training loss and validation loss decreased over the 10 epochs. This means CNN Model 2 was making fewer errors as training continued.

The validation loss decreased steadily and this suggests that the model did not show a strong sign of overfitting. Since Model 2 used dropout, it was able to train in a more regularized way. This result supports the idea that dropout helped the second model generalize more smoothly.

```
In [21]: # Evaluate the 2nd model on the test set
test_loss_2, test_accuracy_2 = cnn_model_2.evaluate(X_test, y_test_cat)

print(f"Model 2 Test Loss: {test_loss_2:.4f}")
print(f"Model 2 Test Accuracy: {test_accuracy_2:.4f}")
```

313/313 ————— 3s 11ms/step - accuracy: 0.6872 - loss: 0.8875
Model 2 Test Loss: 0.8875
Model 2 Test Accuracy: 0.6872

CNN Model 2 Classification Report

CNN Model 2 achieved a test accuracy of about 68.7%.

The strongest results were for automobile, frog, truck, ship, and airplane. For example, automobile had strong precision and recall, meaning the model was usually accurate when predicting automobiles and also found most of the actual automobile images.

The model struggled more with classes like cat, bird, dog, and deer. This is similar to CNN Model 1 and makes sense because these animal classes can look visually similar in small 32 by 32 images. The cat class had the lowest performance, with an f1 score of about 0.44, which shows that the model had difficulty separating cats from other similar classes.

Overall, Model 2 performed better than Model 1. It did not greatly improve the overall test accuracy, but it still learned better image patterns from the dataset.

```
In [22]: # Predict on test set for CNN Model 2
y_pred_prob_2 = cnn_model_2.predict(X_test)

# Convert predicted probabilities into class labels
y_pred_2 = np.argmax(y_pred_prob_2, axis = 1)

# Convert y_test into a one dimensional array
y_test_labels_2 = y_test.flatten()

# Evaluate model performance using classification report
print(classification_report(y_test_labels_2, y_pred_2, target_names = class_names))
```

313/313	4s 11ms/step			
	precision	recall	f1-score	support
airplane	0.74	0.73	0.73	1000
automobile	0.87	0.75	0.81	1000
bird	0.57	0.58	0.57	1000
cat	0.52	0.38	0.44	1000
deer	0.62	0.61	0.62	1000
dog	0.52	0.67	0.59	1000
frog	0.77	0.77	0.77	1000
horse	0.73	0.73	0.73	1000
ship	0.79	0.84	0.81	1000
truck	0.76	0.82	0.79	1000
accuracy			0.69	10000
macro avg	0.69	0.69	0.69	10000
weighted avg	0.69	0.69	0.69	10000


```
In [24]: # Compare test results for both CNN models
results = pd.DataFrame({
    "Model": ["CNN Model 1", "CNN Model 2"],
    "Test Loss": [test_loss_1, test_loss_2],
    "Test Accuracy": [test_accuracy_1, test_accuracy_2]
})

results
```

```
Out[24]:
```

	Model	Test Loss	Test Accuracy
0	CNN Model 1	1.008080	0.6664
1	CNN Model 2	0.887527	0.6872

Comparison of CNN Model 1 and CNN Model 2

The table compares the test loss and test accuracy for both CNN models. CNN Model 1 achieved a test accuracy of about 66.6%, while Model 2 achieved a test accuracy of about 68.7%.

Both models performed at a similar level, but CNN Model 2 was slightly stronger. It had a higher test accuracy and a lower test loss. This suggests that adding a third convolutional layer and dropout helped the model generalize a little better.

The difference between the two models was small, so the improvement was not major. However, Model 2 showed that regularization can help reduce overfitting and improve test performance slightly.

Sample Predictions

The sample predictions show how CNN Model 2 classified individual images from the test set. Each image displays the actual class label and the predicted class label.

Most of the examples shown were classified correctly, such as ship, airplane, automobile, frog, and truck. However, there were also incorrect predictions where an actual frog was predicted as a deer and a cat was predicted as a dog. Even though the model learned useful visual patterns, it still confused classes when the images are small, blurry, or have similar shapes and colors.

This section gives a more visual understanding of the model's performance. The CNN was able to correctly classify many images, but it still made mistakes on some examples, which is consistent with the classification report and confusion matrix.

```
In [27]: y_pred_prob_2 = cnn_model_2.predict(X_test)
y_pred_2 = np.argmax(y_pred_prob_2, axis = 1)

plt.figure(figsize = (12, 8))

for i in range(12):
    plt.subplot(3, 4, i + 1)
    plt.imshow(X_test[i])

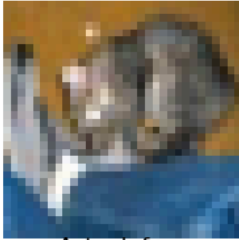
    actual_label = class_names[y_test_labels[i]]
    predicted_label = class_names[y_pred_2[i]]

    plt.title(f"Actual: {actual_label}\nPredicted: {predicted_label}")
    plt.axis("off")

plt.show()
```

313/313 ————— 3s 9ms/step

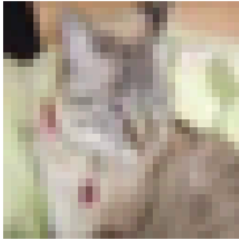
Actual: cat
Predicted: dog



Actual: frog
Predicted: deer



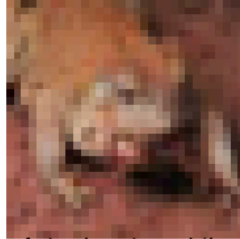
Actual: cat
Predicted: cat



Actual: ship
Predicted: ship



Actual: frog
Predicted: frog



Actual: automobile
Predicted: automobile



Actual: ship
Predicted: ship



Actual: automobile
Predicted: automobile



Actual: airplane
Predicted: cat



Actual: airplane
Predicted: airplane



Actual: frog
Predicted: frog



Actual: truck
Predicted: truck



Final Analysis and Conclusion

The goal of this project was to build a convolutional neural network that could classify CIFAR 10 images into 10 different object categories. Since each image belongs to one class, this was a supervised multi class classification problem.

Two CNN models were created and compared. CNN Model 1 was a basic CNN with two convolutional layers and two pooling layers. It was used as the baseline model. CNN Model 2 added a third convolutional layer and a dropout layer to see if a slightly deeper and more regularized model would perform better.

CNN Model 1 achieved a test accuracy of about 66.6%, while CNN Model 2 achieved a test accuracy of about 68.7%. The results were close, but CNN Model 2 performed slightly better in both test accuracy and test loss. This shows that adding dropout and an extra convolutional layer helped the model generalize a little better, although the improvement was not major.

Both models learned useful visual patterns from the CIFAR 10 images. They performed better on classes such as ship, truck, automobile, frog, and horse. They struggled more with visually similar animal classes such as cat, dog, bird, and deer. This makes sense because CIFAR 10 images are small, and some classes share similar shapes, colors, and textures.

Overall, the project showed how CNNs can be used for image classification. It also showed that adding dropout can help reduce overfitting and improve generalization, even if the improvement in accuracy is small.

```
In [32]: !jupyter nbconvert --to webpdf --allow-chromium-download "Ashton_ML_Final.ipynb"
```

```
[NbConvertApp] Converting notebook Ashton_ML_Final.ipynb to webpdf
[NbConvertApp] WARNING | Alternative text is missing on 8 image(s).
[NbConvertApp] Building PDF
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 593817 bytes to Ashton_ML_Final.pdf
```

```
In [ ]:
```